

Pandas Practice Questions

From Absolute Basics to Intermediate Level

200+ hands-on questions across 10 sections, grouped by difficulty.

Work through them with a real dataset (or make one up) and a Jupyter/Colab notebook.

Try to solve each question in one line of pandas before checking documentation.

Contents

- 1. Getting Started: Series & DataFrames (Basics)
- 2. Reading & Writing Data (Basics)
- 3. Inspecting & Exploring Data (Basics)
- 4. Selecting, Indexing & Filtering (Basics → Intermediate)
- 5. Handling Missing Data (Intermediate)
- 6. Data Cleaning & Transformation (Intermediate)
- 7. GroupBy, Aggregation & Pivot Tables (Intermediate)
- 8. Merging, Joining & Concatenating (Intermediate)
- 9. Dates, Times & String Operations (Intermediate)
- 10. Mini Capstone Challenges (Applied Practice)

1. Getting Started: Series & DataFrames

Goal: get comfortable creating pandas objects from scratch and understanding their basic anatomy (index, values, dtype).

Very Basic

1. Import pandas with its standard alias.
2. Create a pandas Series from the list [10, 20, 30, 40, 50].
3. Create a Series from a Python dictionary {'a': 1, 'b': 2, 'c': 3}.
4. Create a Series with a custom index: values [5, 10, 15] and index ['x', 'y', 'z'].
5. Find the data type (dtype) of a Series containing [1, 2, 3].
6. Create a DataFrame from a dictionary of lists with columns 'Name' and 'Age'.
7. Create a DataFrame from a list of lists, specifying column names manually.
8. Print the shape (rows, columns) of a DataFrame.
9. Print the column names of a DataFrame.
10. Print the index (row labels) of a DataFrame.

Basic

11. Create a DataFrame with 5 rows and 3 columns using NumPy random numbers.
12. Convert a Series into a DataFrame.
13. Convert a DataFrame column into a Series.
14. Rename the columns of an existing DataFrame.
15. Set an existing column as the DataFrame's index.
16. Reset a DataFrame's index back to default integers.
17. Check the memory usage of a DataFrame using .info().
18. Get the number of dimensions of a DataFrame using .ndim.
19. Create a DataFrame with a DatetimeIndex ranging over 7 days starting today.
20. Create an empty DataFrame and then add columns to it one by one.

2. Reading & Writing Data

Goal: practice moving data in and out of pandas. Use any CSV/Excel file you have, or create small sample files yourself.

Basic

21. Read a CSV file into a DataFrame using `pd.read_csv()`.
22. Read only the first 100 rows of a large CSV file.
23. Read a CSV file while specifying custom column names.
24. Read a CSV file that uses a semicolon (;) as the delimiter.
25. Save a DataFrame to a CSV file without writing the index column.
26. Read an Excel file into a DataFrame using `pd.read_excel()`.
27. Read a specific sheet from a multi-sheet Excel file.
28. Save a DataFrame to an Excel file with a custom sheet name.
29. Read a JSON file into a DataFrame using `pd.read_json()`.
30. Convert a DataFrame to a JSON string using `.to_json()`.

Intermediate

31. Read a CSV file while parsing a specific column as datetime using `parse_dates`.
32. Read a CSV file while treating certain string values (like 'NA', '-', '?') as missing values.
33. Read a large CSV file in chunks using the `chunksize` parameter and process each chunk.
34. Read a CSV file directly from a URL.
35. Write a DataFrame to a CSV file using a different encoding (e.g., 'utf-8-sig').
36. Read a CSV file while skipping the first 2 rows and the last 3 rows.

3. Inspecting & Exploring Data

Goal: build the habit of always exploring a new dataset before touching it — shape, types, summary stats, and a quick look.

Very Basic

- 37. Display the first 5 rows of a DataFrame using `.head()`.
- 38. Display the last 10 rows of a DataFrame using `.tail()`.
- 39. Get a random sample of 5 rows using `.sample()`.
- 40. Print summary statistics (mean, std, min, max, etc.) using `.describe()`.
- 41. Get the data types of every column using `.dtypes`.
- 42. Count the number of non-null values in each column using `.count()`.
- 43. Find the number of unique values in a specific column using `.nunique()`.
- 44. List all unique values in a specific column using `.unique()`.
- 45. Count how many times each value appears in a column using `.value_counts()`.
- 46. Check for duplicate rows in a DataFrame using `.duplicated()`.

Basic

- 47. Get a concise summary of the DataFrame (columns, non-null counts, dtypes) using `.info()`.
- 48. Find the column with the most missing values.
- 49. Compute the correlation matrix of all numeric columns using `.corr()`.
- 50. Find the minimum and maximum values of a numeric column.
- 51. Calculate the mean, median, and mode of a numeric column.
- 52. Sort a DataFrame by a single column in descending order.
- 53. Sort a DataFrame by multiple columns, with different sort orders for each.
- 54. Find the top 5 rows with the highest values in a given column using `.nlargest()`.
- 55. Find the bottom 5 rows with the lowest values in a given column using `.nsmallest()`.
- 56. Transpose a DataFrame using `.T`.

4. Selecting, Indexing & Filtering

Goal: master `.loc`, `.iloc`, boolean masks, and conditional selection — the single most-used skill in daily pandas work.

Basic

- 57. Select a single column from a DataFrame as a Series.
- 58. Select multiple columns from a DataFrame as a new DataFrame.
- 59. Select the first 5 rows using `.iloc`.
- 60. Select a specific row by its label using `.loc`.
- 61. Select a specific cell value using both `.loc` and `.iloc`.
- 62. Select rows where a numeric column is greater than a given value.
- 63. Select rows where a column's value matches one of several options using `.isin()`.
- 64. Select rows where a string column is NOT equal to a specific value.
- 65. Select all rows and only specific columns using `.loc`.
- 66. Drop a specific column from a DataFrame using `.drop()`.
- 67. Drop a specific row from a DataFrame using `.drop()`.

Intermediate

- 68. Filter rows using two conditions combined with AND (&).
- 69. Filter rows using two conditions combined with OR (|).
- 70. Filter rows using the `.query()` method instead of boolean indexing.
- 71. Select rows where a column value falls within a given range (between two numbers).
- 72. Use `.loc` with a boolean condition to update values in a specific column.
- 73. Select a subset of rows and columns simultaneously using `.loc` with conditions.
- 74. Use `.at[]` to get/set a single scalar value efficiently.
- 75. Filter a DataFrame to remove rows containing a specific substring in a text column.
- 76. Select every other row of a DataFrame using slicing.
- 77. Reorder the columns of a DataFrame into a custom sequence.

5. Handling Missing Data

Goal: confidently detect, remove, and fill missing values — a critical step in almost every real dataset.

Basic

- 78. Check which cells in a DataFrame contain missing values using `.isna()`.
- 79. Count the total number of missing values in each column.
- 80. Count the total number of missing values in the entire DataFrame.
- 81. Drop all rows that contain at least one missing value using `.dropna()`.
- 82. Drop all columns that contain at least one missing value.
- 83. Fill all missing values in a DataFrame with a constant, such as 0.
- 84. Fill missing values in a specific column with the column's mean.
- 85. Fill missing values in a specific column with the column's median.

Intermediate

- 86. Drop rows only if ALL values in that row are missing (not just some).
- 87. Drop rows that have fewer than a minimum number of non-null values using the `thresh` parameter.
- 88. Forward-fill missing values (propagate the last valid value forward).
- 89. Backward-fill missing values (use the next valid value to fill gaps).
- 90. Fill missing values differently for different columns using a dictionary.
- 91. Interpolate missing numeric values using linear interpolation.
- 92. Replace a specific placeholder value (like -999 or 'unknown') with NaN across the DataFrame.

6. Data Cleaning & Transformation

Goal: reshape raw, messy columns into clean, analysis-ready ones using `apply`, `map`, and vectorized operations.

Basic

- 93. Rename a single column of a DataFrame without affecting the others.
- 94. Convert a column's data type from object (string) to numeric using `pd.to_numeric()`.
- 95. Convert a numeric column to string type using `.astype(str)`.
- 96. Strip leading/trailing whitespace from all values in a text column.
- 97. Convert all text in a column to lowercase.
- 98. Replace specific values in a column using `.replace()`.
- 99. Create a new column that is the result of a calculation on two existing columns.
- 100. Round all values in a numeric column to 2 decimal places.
- 101. Remove duplicate rows from a DataFrame using `.drop_duplicates()`.
- 102. Remove duplicate rows based on a subset of specific columns only.

Intermediate

- 103. Apply a custom function to every value in a column using `.apply()`.
- 104. Apply a custom function to every row of a DataFrame using `.apply(axis=1)`.
- 105. Use `.map()` to translate category codes (e.g., 'M'/'F') into full labels ('Male'/'Female').
- 106. Bin a continuous numeric column into discrete categories using `pd.cut()`.
- 107. Bin a numeric column into equal-sized quantile groups using `pd.qcut()`.
- 108. Use `.applymap()` (or `DataFrame.map` in newer pandas) to apply a function to every cell in a DataFrame.
- 109. Create a new column using conditional logic with `np.where()`.
- 110. Create a new column with multiple conditional categories using `pd.cut()` or nested `np.select()`.
- 111. Use `.clip()` to cap outlier values in a numeric column between a minimum and maximum.
- 112. Normalize a numeric column to a 0-1 range (min-max scaling) using a custom formula.

7. GroupBy, Aggregation & Pivot Tables

Goal: this is where pandas becomes powerful for real analysis — practice until groupby feels automatic.

Basic

- 113. Group a DataFrame by a single categorical column and compute the mean of a numeric column.
- 114. Group by a column and count the number of rows in each group using `.size()`.
- 115. Group by a column and compute multiple statistics (mean, sum, count) at once using `.agg()`.
- 116. Group by two columns simultaneously and compute the mean of a numeric column.
- 117. Find the group with the highest average value for a given numeric column.

Intermediate

- 118. Group by a column and apply different aggregation functions to different columns using a dictionary passed to `.agg()`.
- 119. Use `.transform()` within a groupby to add a column showing each group's mean alongside the original rows.
- 120. Create a pivot table showing the average of one column, grouped by two other columns, using `pd.pivot_table()`.
- 121. Create a pivot table with a custom aggregation function (e.g., sum instead of mean).
- 122. Use `.pivot()` to reshape a DataFrame from long format to wide format.
- 123. Use `pd.melt()` to reshape a DataFrame from wide format to long format.
- 124. Group data by a datetime column resampled to monthly frequency using `.resample()`.
- 125. Apply a custom aggregation function to a groupby object using `.agg(custom_function)`.
- 126. Filter groups after a groupby based on a group-level condition using `.filter()`.
- 127. Rank values within each group using `.rank()` after a groupby.

8. Merging, Joining & Concatenating

Goal: practice combining multiple DataFrames the way you would combine tables from different sources in a real project.

Basic

- 128.** Concatenate two DataFrames vertically (stack rows) using `pd.concat()`.
- 129.** Concatenate two DataFrames horizontally (side by side) using `pd.concat(axis=1)`.
- 130.** Merge two DataFrames on a common column using `pd.merge()` with an inner join.
- 131.** Merge two DataFrames using a left join, keeping all rows from the first DataFrame.
- 132.** Merge two DataFrames using a right join.
- 133.** Merge two DataFrames using an outer join to keep all rows from both.

Intermediate

- 134.** Merge two DataFrames on columns with different names using `left_on` and `right_on`.
- 135.** Merge two DataFrames on their indexes rather than columns.
- 136.** Identify which rows in a merge came from only the left, only the right, or both DataFrames using the `indicator` parameter.
- 137.** Join two DataFrames using `.join()` instead of `.merge()`.
- 138.** Handle overlapping column names during a merge by specifying custom suffixes.
- 139.** Concatenate multiple DataFrames from a list of files (e.g., simulate reading and combining monthly CSVs).

9. Dates, Times & String Operations

Goal: get fluent with the `.dt` and `.str` accessors — common in almost every real-world dataset.

Basic

- 140. Convert a text column to datetime using `pd.to_datetime()`.
- 141. Extract the year, month, and day from a datetime column into separate columns.
- 142. Extract the day of the week (e.g., Monday, Tuesday) from a datetime column.
- 143. Filter rows where a date column falls within a specific date range.
- 144. Calculate the number of days between two datetime columns.
- 145. Check if a string column contains a specific substring using `.str.contains()`.
- 146. Split a string column into multiple columns using `.str.split()`.
- 147. Combine two string columns into one using string concatenation.
- 148. Count the number of characters in each string in a column using `.str.len()`.
- 149. Replace a substring within a text column using `.str.replace()`.

Intermediate

- 150. Set a datetime column as the DataFrame index and select data by year using slicing.
- 151. Calculate a 7-day rolling average of a numeric column using `.rolling()`.
- 152. Shift a numeric column forward or backward by one period using `.shift()` (useful for calculating differences over time).
- 153. Extract a substring from each value in a column using regular expressions with `.str.extract()`.
- 154. Check if a string column matches a specific pattern using `.str.match()` with a regex.
- 155. Convert a column of full names into separate first-name and last-name columns.
- 156. Calculate the cumulative sum of a numeric column over time using `.cumsum()`.
- 157. Find the difference between consecutive rows in a numeric column using `.diff()`.

10. Mini Capstone Challenges

Goal: combine everything above. Pick any dataset you like (e.g., a sales, movies, or survey dataset) and try these end-to-end tasks.

- 158.** Load a raw CSV dataset, inspect it, and write a short summary of its shape, column types, and missing values.
- 159.** Clean a messy dataset: fix column names, handle missing values, remove duplicates, and correct data types.
- 160.** Find the top 10 records by a given metric (e.g., highest sales, highest score) after filtering out invalid rows.
- 161.** Group a dataset by a categorical column and produce a summary table of counts, means, and totals.
- 162.** Create a pivot table that answers a specific business question (e.g., 'average revenue per region per month').
- 163.** Merge two related datasets (e.g., orders and customers) and answer a question that requires both (e.g., total spend per customer).
- 164.** Engineer at least 3 new features (columns) from existing data using apply, map, or vectorized operations.
- 165.** Detect and handle outliers in a numeric column using IQR or z-score, then compare summary stats before and after.
- 166.** Convert a raw date column into useful time-based features (year, month, weekday, is_weekend) and analyze trends by time.
- 167.** Export your final cleaned and transformed DataFrame to a new CSV file, ready for analysis or visualization.

Tip: once you can solve all of the above without looking anything up, you're solidly at an intermediate-to-advanced pandas level. Next steps: practice with pandas' MultiIndex, method chaining style, performance optimization (vectorization vs. apply), and integrating pandas with matplotlib/seaborn for visualization.